

DSA PRACTICAL 2

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches.

Practical Aim:

To implement linear and binary search using both iterative and recursive approaches.

Problem 1:

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Part 1(Linear search):

```
#include<iostream>

#include<string>

using namespace std;

int main()

{

    int ar[10]={1001,1002,1003,1004,1005,1006,1007,1008,1009,1010};

    int mid=0,start=0,last=9;

    int plate=0,key=0;


    cout<<"Enter the plate number : ";

    cin>>plate;

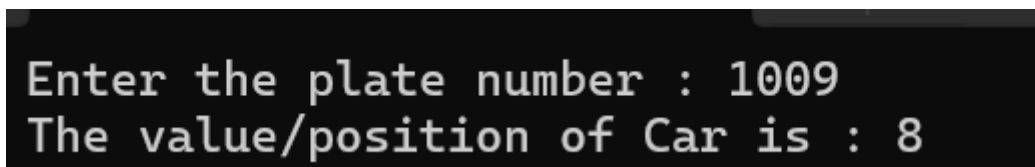
    for(int i=0;i<=last;i++)

    {
```

DSA PRACTICAL 2

```
mid=(start+last)/2;
if(ar[mid]>plate)
{
    last=mid-1;
}
else if(ar[mid]<plate)
{
    start=mid+1;
}
else
{
    key=mid;
    // cout<<"The number is at : "<<mid+1<<"position";
}
}
cout<<"The value/position of Car is : "<<key;
}
```

Screenshot :



```
Enter the plate number : 1009
The value/position of Car is : 8
```

Description:

Searching car plate number with the help of linear search in a known plates.

DSA PRACTICAL 2

Part 2(Binary search vehicle number):

```
#include<iostream>

#include<string>

using namespace std;

int main()

{

int plate[100]={0}; // the capacity of parking is 100


int n=0;//number plate to find

int j=0;//temporary stores how many cars

cout<<"Enter the car number plates and press 0 for exit : ";

for(int i=0;i<100;i++)

{

    cin>>plate[i];

    if(plate[i]==0)

    {

        j=i;

        break;

    }

}

cout<<"Enter the plate number of the car to find: ";

cin>>n;

for(int i=0;i<j;i++)

{

    if(plate[i]==n)

    {

        cout<<"The car with plate number "<<n<<"was at "<<i+1<<"th position"<<endl;

    }

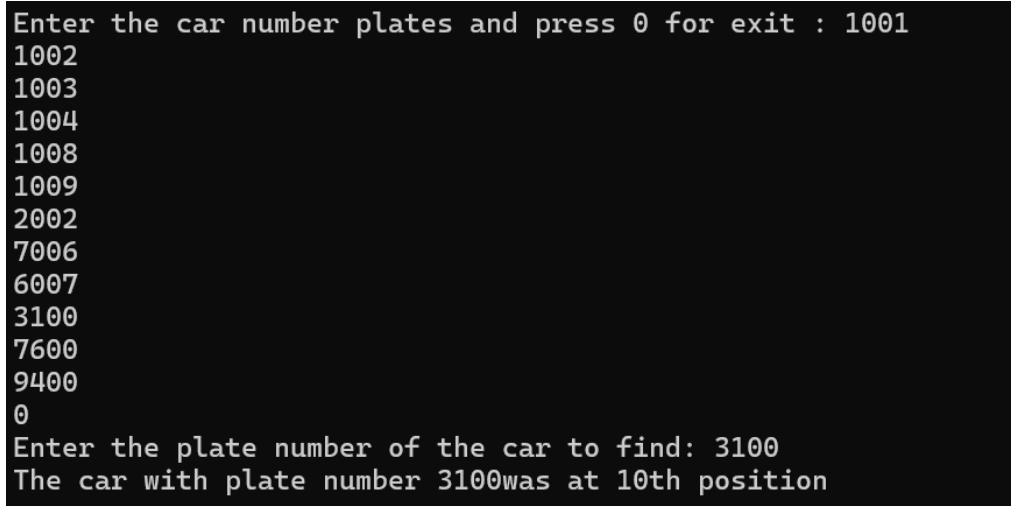
}
```

25DCE110

DSA PRACTICAL 2

```
}  
  
return 0;  
  
}
```

Screenshot :



```
Enter the car number plates and press 0 for exit : 1001  
1002  
1003  
1004  
1008  
1009  
2002  
7006  
6007  
3100  
7600  
9400  
0  
Enter the plate number of the car to find: 3100  
The car with plate number 3100was at 10th position
```

Description:

Enter plate numbers and searching them using binary search.

DSA PRACTICAL 2

Problem 2

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code.

Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

Part 1(Binary search):

```
#include<iostream>

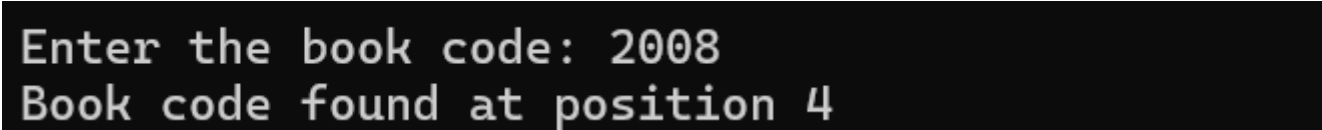
using namespace std;

int main()
{
    int plate[]={2001,2003,2005,2008,2011,2022,2026};
    int size=7;
    int n;
    cout<<"Enter the book code: ";
    cin>>n;
    int low=0,high=size-1;
    while(low<=high)
    {
        int mid=(low+high)/2;
        if(plate[mid]==n)
        {
            cout<<"Book code found at position "<<mid+1;
            return 0;
        }
    }
    25DCE110
```

DSA PRACTICAL 2

```
if(n<plate[mid])  
    high=mid-1;  
else  
    low=mid+1;  
}  
cout<<"Book code not found.";  
return 0;  
}
```

Screenshot :



```
Enter the book code: 2008  
Book code found at position 4
```

Description:

Searching books in library using its code with the help of binary search .

DSA PRACTICAL 2

Part 2(Binary search with recursion):

```
#include<iostream>

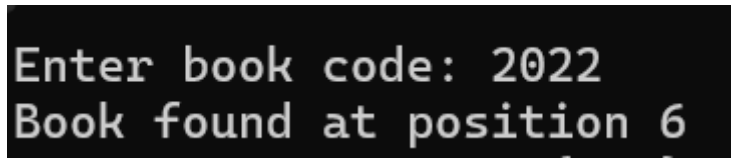
using namespace std;

int search(int a[],int low,int high,int key)
{
    if(low>high)
        return -1;
    int mid=(low+high)/2;
    if(a[mid]==key)
        return mid;
    if(key<a[mid])
        return search(a,low,mid-1,key);
    else
        return search(a,mid+1,high,key);
}

int main()
{
    int a[]={2001,2003,2005,2008,2011,2022,2026};
    int key;
    cout<<"Enter book code: ";
    cin>>key;
    int ans=search(a,0,6,key);//(array,low,high,value/key)
    if(ans== -1)
        cout<<"Book not found";
    else
        cout<<"Book found at position "<<ans+1;
    return 0;}
25DCE110
```

DSA PRACTICAL 2

Screenshot :



```
Enter book code: 2022
Book found at position 6
```

Description:

Searching books in library from book code, but binary search in recursive function.

Conclusion:

The practical successfully implemented linear search and binary search using iterative and recursive approaches. Linear search checks elements sequentially, while binary search repeatedly divides the sorted list into smaller parts to locate the required element efficiently. The practical also demonstrated that binary search requires the input data to be sorted. The iterative approaches use loops, whereas the recursive approaches use repeated function calls. Overall, binary search is more efficient than linear search for large sorted datasets, with a time complexity of $O(\log n)$ compared with $O(n)$ for linear search.